FINAL YEAR PROJECT REPORT

# LeadBot

## *An AI lead-qualification chatbot that any small business can set up for itself*

A chatbot product that a business signs up for, trains on its own information in a guided setup with a live preview, and adds to its website with one line of code. It answers in English and Roman Urdu, captures and scores leads, and gives each owner their own pipeline, audit log and ways to train the bot.


| | |
|---|---|
| Team | **Mohammed Maarij**<br>19252105<br>**Abdul Manan Khan**<br>19252165<br>**Muhammad Ali Qureshi**<br>19252094<br>**Muhammad Hamza Younus**<br>19252201 |
| Supervisor | **Muhammad Siddique** |
| Department | **BSCS** |
| University | **Federal Urdu University of Arts, Science and Technology** |
| Date | **October 2026** |
| Live demo | **https://leadbot-g3d4.onrender.com** |
| Source code | **https://github.com/MAliQ-pak/leadbot** |

# Contents


Abstract

Team and contributions

1 Introduction

2 Background

3 Requirements

4 System design

5 Implementation

6 Testing and evaluation

7 Deployment

8 Limitations and future work

9 Conclusion

References

Appendix A: Viva questions and answers

# Abstract

LeadBot is a working chatbot product for small businesses. A business signs up, picks one of eight industry templates (from solar installer and shoe store to clinic, restaurant and salon), teaches the bot its own prices and policies in a five-step setup wizard while testing it in a live preview, and then adds it to its own website with one line of code. The bot answers customer questions only from that business's knowledge, collects lead details in conversation, and scores each lead as hot, warm or cold with a stated reason. The system is deployed at a public link and demonstrated on two fictional businesses, a solar installer (BrightPath Solar) and a shoe store (Stride Shoes).

Replies are produced by retrieval-augmented generation: a TF-IDF retriever over character n-grams finds the relevant topics in that business's knowledge base, and a large language model (Google Gemini, or Anthropic Claude) answers only from them. A rule-based offline bot takes over whenever no AI is available, so the demo never depends on internet or payment. Lead scores blend explainable rules with a logistic-regression classifier.

Each owner has a private dashboard: a deals pipeline with stage names for their industry, leads by temperature, full conversations, notes, an audit log, and four ways to train the bot (Q&A pairs, unanswered questions, uploaded documents, imported web pages). Every row of data belongs to one business, and an owner can only reach their own. In testing, the classifier reached 92% accuracy in 5-fold cross-validation on a 62-example illustrative dataset; the AI answered in English and Roman Urdu without inventing prices or accepting a fake discount; and a new shoe shop was signed up, trained, tested and embedded on a separate website, with its first real lead arriving on its own pipeline.

# Team and contributions

LeadBot was built by a team of four. Each member owned one area of the system end to end, from design to testing, and presents it at the viva. All four contributed to the requirements, the joint testing sessions and this report.

| Member | Roll number | Area owned | Main files |
|---|---|---|---|
| Mohammed Maarij | 19252105 | Product pages, setup wizard, chat window and embed code, demo websites, user-interface design | `static/landing.html, auth.html, setup.*, bot.*, widget.js, product.css, demo/, static/fonts/` |
| Abdul Manan Khan | 19252165 | AI and retrieval: RAG, Gemini / Claude, prompt, offline bot, industry templates, knowledge-base training | `app/rag.py, app/llm.py, app/ingest.py, app/business.py, static/admin-kb.js, kb/business.md` |
| Muhammad Ali Qureshi | 19252094 | Backend, accounts and data separation, dashboard (pipeline, audit log), security, database and deployment | `app/main.py, app/db.py, app/auth.py, app/seed.py, static/admin.*, admin-pipeline.js, render.yaml` |
| Muhammad Hamza Younus | 19252201 | Lead scoring, the classifier, testing and evaluation | `app/scoring.py, app/classifier.py, train_classifier.py, data/labeled_leads.csv` |

# 1 Introduction

## 1.1 Problem

Small businesses lose sales because website visitors ask questions outside office hours, get no answer, and leave without sharing contact details. When enquiries do arrive, the owner must read each one to decide who is a serious buyer. In Pakistan this is harder: customers often write in Roman Urdu, which simple keyword bots handle poorly. And a small business has no developer, so a chatbot is only useful if the owner can set it up and keep it up to date without help.

## 1.2 Aim

Build a chatbot product that any small business can sign up for, train on its own information and put on its own website, which answers questions from that information, captures and qualifies leads automatically, and hands the owner a ranked list of who to call first.

## 1.3 Objectives

1. Answer visitor questions only from the business's knowledge base, never inventing prices or policies.
2. Support English and Roman Urdu.
3. Extract lead details (name, phone, email, need, budget, timeline) from natural conversation.
4. Score each lead 0-100 with a hot, warm or cold tier and a human-readable reason, combining rules with a trained classifier.
5. Give the owner a dashboard to manage leads and to train the bot without editing code.
6. Keep working with no internet or AI access, so a demonstration cannot fail.
7. Deploy the system at a public link with basic security.
8. Let any business sign up, set up its own bot from an industry template, test it in a live preview and embed it on its website, with every business's data kept separate.

## 1.4 Scope

The first version served one business, configured by editing a Markdown file. After it was demonstrated, our supervisor asked how the bot could be trained quickly for a different company, for example a shoe shop. Answering that question properly turned LeadBot into a product: public sign-up, industry templates, a setup wizard with a live preview, embed code, and one database shared safely by many businesses.

The demo businesses, BrightPath Solar and Stride Shoes, are fictional. LeadBot covers a website chat, not WhatsApp or phone channels. Each business has one owner login. CRM integration was built as an optional webhook but is switched off for this project, with leads managed in the dashboard instead.

## 1.5 Report structure

The Team and contributions section lists who built each part. Section 2 reviews the background. Sections 3 to 5 cover requirements, design and implementation. Section 6 evaluates the system, Section 7 describes deployment, and Sections 8 and 9 discuss limitations and conclude. Appendix A prepares answers for the viva.

# 2 Background

LeadBot combines four established ideas: retrieval-augmented generation for grounded answers, classic TF-IDF retrieval for finding the right text, lead scoring for sales prioritisation, and multi-tenancy, where one system serves many customers. References marked [R#] are listed in the References section and must be checked before submission.

## 2.1 Website chatbots

Early website bots followed fixed decision trees or keyword rules: predictable, but brittle when visitors phrase things differently. Large language models (LLMs) understand free text and reply fluently, but on their own they can state plausible facts that are wrong, such as an invented price. For a sales bot, that risk is unacceptable.

## 2.2 Retrieval-augmented generation (RAG)

RAG retrieves relevant passages first and asks the model to answer from them [R1]. The model's knowledge of the business then comes only from text the owner controls, and updating the bot means updating that text, not retraining a model. This is also what makes a multi-business product practical: a new business needs new text, not a new model. LeadBot adds an explicit rule: if the retrieved text does not cover the question, the bot says it is not sure and offers a human callback.

## 2.3 TF-IDF retrieval

TF-IDF weights a term by how often it appears in a passage, discounted by how common it is across all passages [R2]. Cosine similarity between the question's vector and each passage's vector ranks the passages. LeadBot uses character n-grams (3 to 5 characters) instead of whole words, so spelling variants common in Roman Urdu, such as "qeemat" and "keemat" (price), still share most of their n-grams. The vectoriser comes from scikit-learn [R3].

Embedding-based retrieval with a vector database is the modern alternative. It handles synonyms better but needs an embedding model or API. For a knowledge base of tens of topics, TF-IDF is fast, free, offline and explainable, which suits this project.

## 2.4 LLM APIs

Hosted LLMs are called over HTTPS with a secret API key. LeadBot supports two providers: Google Gemini, which offers a free tier [R4], and Anthropic Claude, which is paid per token [R5]. Both are asked to return JSON containing the reply and any lead details, which the server then validates.

## 2.5 Lead qualification and scoring

Sales teams rank leads so effort goes to the most likely buyers. Rule-based scoring adds points for signals such as a shared phone number or a stated budget; it is transparent but its weights are guesses. Predictive scoring learns weights from past outcomes, often with logistic regression [R6],

whose coefficients can still be read and explained. LeadBot uses both and shows each score separately.

## 2.6 Multi-tenant software

Software-as-a-service products such as hosted chat and CRM tools serve many customer organisations, called tenants, from one system. The simplest design keeps every tenant's rows in shared tables and tags each row with a tenant ID; every query then filters by that ID. It is cheap to run and easy to back up, but the separation is only as strong as the code that applies the filter, so that code must take the tenant from the logged-in session, never from the request. LeadBot follows this design.

# 3 Requirements

The system serves two users: the **website visitor**, who chats anonymously on a business's website, and the **business owner**, who signs up, sets up the bot and logs into the dashboard.

## 3.1 Functional requirements

| ID | Requirement | User |
|---|---|---|
| FR1 | Chat embeddable on any website with one script tag (or an iframe, or a direct link) | Visitor |
| FR2 | Answer questions only from the business's knowledge base; say "not sure" otherwise | Visitor |
| FR3 | Reply in the visitor's language (English or Roman Urdu) | Visitor |
| FR4 | Extract name, phone, email, need, budget and timeline from chat | System |
| FR5 | Score each lead 0-100 with a tier (hot, warm, cold) and a reason | System |
| FR6 | List, filter and search leads; view full conversations | Owner |
| FR7 | Move deals through six stages on a pipeline board, with stage names for the industry; add notes | Owner |
| FR8 | Train the bot: Q&A pairs, unanswered questions, documents, web pages, main text | Owner |
| FR9 | Test what the bot would retrieve for any question | Owner |
| FR10 | Optional push of warm and hot leads to a CRM webhook | System |
| FR11 | Audit log of owner actions and key system events, filterable by type | Owner |
| FR12 | Sign up a new business with an owner account; log in and out | Owner |
| FR13 | Start from an industry template (eight, from solar installer and shoe store to clinic, restaurant and salon) | Owner |
| FR14 | Set up the bot in guided steps: business, knowledge, look and greeting, lead questions, install | Owner |
| FR15 | Test the bot in a live preview while setting it up, without the test chats counting as leads | Owner |
| FR16 | Keep every business's leads, chats, knowledge and settings separate from every other's | System |

## 3.2 Non-functional requirements

| ID | Requirement | How it is met |
|---|---|---|
| NFR1 | Works offline | Rule-based bot answers when no AI key, quota or internet is available |
| NFR2 | Explainable | Every score shows its rule reasons and the model's probability |
| NFR3 | Secure accounts | Hashed passwords, signed session cookie, login lockout, every owner request scoped to the owner's business |
| NFR4 | Controlled cost | Limits of 200 AI replies per hour and 30 per conversation |

| ID | Requirement | How it is met |
|---|---|---|
| NFR5 | Responsive | Pages work at phone width (375 px) with no sideways scrolling |
| NFR6 | Fast replies | Typical AI reply in 2-5 s; an overloaded model gets 8 s, then the fallback answers |
| NFR7 | Simple to run | One command locally; one Blueprint file on Render |
| NFR8 | Data survives restarts | PostgreSQL on the live site; SQLite file on a laptop |
| NFR9 | Works on any website | The chat runs in an iframe served by LeadBot, so the host site needs no set-up and cannot break its styles |

# 4 System design

LeadBot is one Python web server (FastAPI) that serves the product pages, every business's chat window and API, and the owners' dashboards. One database holds all businesses, with every row tagged by its business. Keeping everything in one process makes it simple to run, deploy and explain.

## 4.1 Architecture

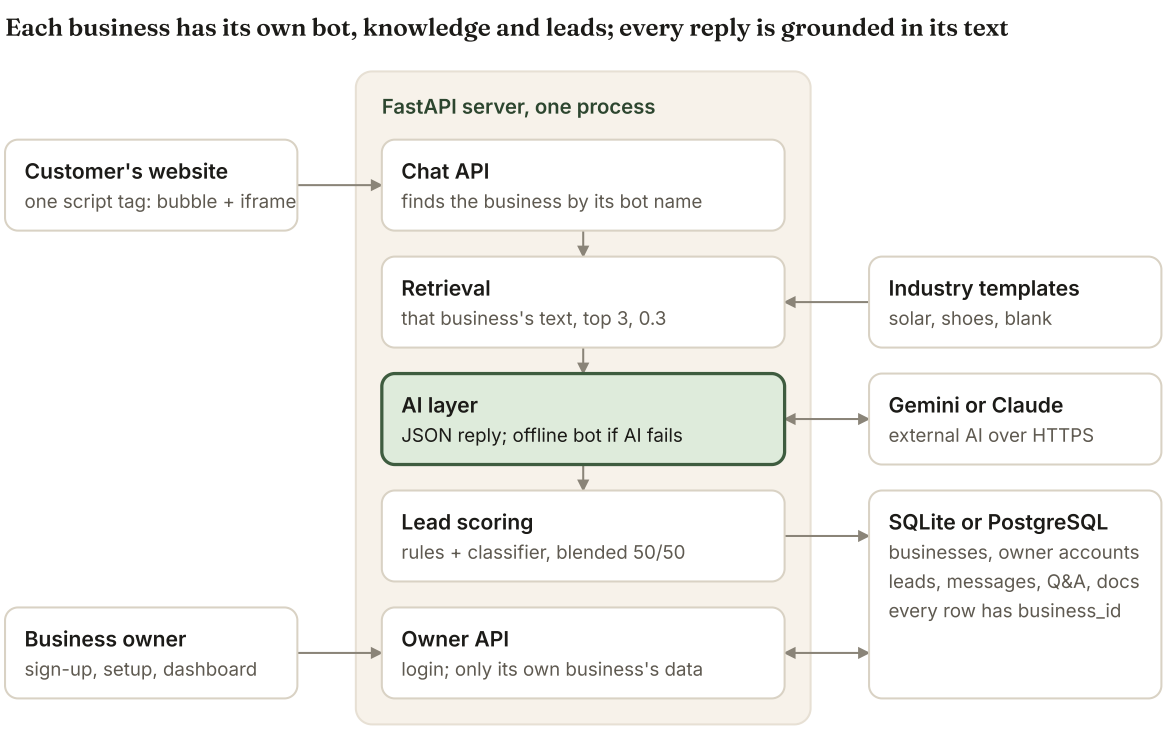



**Figure 1.** LeadBot architecture: one server for every business, with each row of data tagged by its business.

A chat message runs down the server column. The chat API identifies the business from the bot name in the embed code; retrieval then reads only that business's text, Q&A pairs and documents. The owner API identifies the business from the logged-in owner's session instead. If the external AI fails, the AI layer answers with the offline bot.

## 4.2 What happens to one chat message

1. The chat window sends the visitor's message, the business's bot name (for example `stride-shoes`) and a random session ID to `POST /api/chat`.
2. The server finds the business, stores the message and loads the last 12 messages of that session.
3. The retriever scores every topic in that business's knowledge base and keeps the top 3. A short follow-up such as "and the inverter?" is retried together with the previous question.
4. If the best match is below 0.3 and the message is a question, the server treats it as unknown: it logs it as **unanswered** for that business's owner and sends the AI no context, so the AI says it is not

sure.

5. The AI provider (Gemini or Claude) receives the rules, the business's name and lead questions, the details already known about the visitor, the retrieved topics and the conversation. It returns JSON with the reply and any lead details. On any API problem, the offline bot answers instead.
6. The rules and the classifier each score the lead, with the business's own buying-signal words; the blended score sets the tier.
7. The reply, the lead and the score are saved under that business, and the reply is returned to the chat window.

## 4.3 Data model

| Table | Holds | Key columns |
|---|---|---|
| `businesses` | One row per business | slug (bot name), name, industry, website, bot_name, greeting, color, settings (JSON), kb_text, setup_done |
| `users` | Owner accounts | email, password_hash, name, business_id |
| `messages` | Every chat message | business_id, session_id, role (user / assistant), content, created |
| `leads` | One row per visitor session | business_id, name, phone, email, need, budget, timeline, score, rule_score, ml_score, tier, reason, status (deal stage), notes, is_test |
| `kb_faq` | Q&A pairs added by the owner | business_id, question, variants, answer, source (manual / unanswered) |
| `kb_docs` | Uploaded documents and imported web pages | business_id, name, kind (file / url), origin, text |
| `unanswered` | Questions the bot could not answer confidently | business_id, session_id, question, score, status (open / answered / dismissed) |
| `audit` | Owner actions and key system events | business_id, created, actor, kind (lead / kb / auth / system), action, target, detail |

A business's main text is stored in its `kb_text` column, where each `##` heading is one topic. Its `settings` hold the quick replies, the five lead questions, the product and buying-signal words, and the names of the six pipeline stages. New columns are added with `ALTER TABLE` at start-up, so an existing database keeps working after an upgrade: when the multi-business version first started, it created BrightPath Solar as business 1 and assigned it all the older rows.

## 4.4 Technology choices

| Choice | Why |
|---|---|
| Python + FastAPI | Short, readable code; request validation built in |
| SQLite on a laptop, PostgreSQL live | SQLite needs no set-up and works offline; PostgreSQL (a free Neon database) keeps the live site's data across restarts. The same SQL runs on both |
| scikit-learn | TF-IDF retrieval and logistic regression in a few lines |

| Choice | Why |
| --- | --- |
| Gemini / Claude APIs | Strong multilingual replies; Gemini has a free tier |
| Plain HTML, CSS, JavaScript | No build step; every file can be explained line by line |
| An iframe for the chat | Works on any website without cross-site (CORS) set-up; the host page's styles cannot break it |
| Python's `hashlib` and `hmac` | Password hashing and signed cookies with no extra library |
| Render | Free hosting directly from the GitHub repository |

# 5 Implementation

The code is about 2,400 lines of Python (including the demo data) and 2,500 of front-end code, split into small single-purpose files. The table maps each file to its job.

| File | Job |
|---|---|
| `app/main.py` | API routes, pages, the chat pipeline, score blending |
| `app/business.py` | Industry templates and each business's bot settings |
| `app/auth.py` | Owner accounts: password hashing, session cookie, login lockout |
| `app/rag.py` | Knowledge-base loading and TF-IDF retrieval, one index per business |
| `app/llm.py` | Prompt, Gemini and Claude calls, offline bot, limits |
| `app/scoring.py` | Rule-based lead score |
| `app/classifier.py` | Classifier features and prediction |
| `app/ingest.py` | Reading PDF, Word, TXT, Markdown and web pages |
| `app/db.py` | SQLite or PostgreSQL storage |
| `app/seed.py` | The two fictional demo businesses |
| `train_classifier.py` | Trains and evaluates the classifier |
| `static/` | Product pages, setup wizard, chat window, embed loader, dashboard, demo websites |

## 5.1 Product pages, the chat window and the embed code

The **home page** explains the product in four steps and links to the two demo websites. **Sign-up** asks for the business name, the closest industry, an optional website, and the owner's name, email and password; **log-in** asks for email and password.

The **chat window** is one page, `/bot/{bot name}`. It reads the business's public settings (bot name, greeting, brand colour, quick replies), keeps the visitor's session ID in the browser so a page reload restores the chat, and writes every reply as text, never as HTML. A business puts it on its website with one line:

```
<script src=".../static/widget.js" data-bot="stride-shoes" data-color="#2E3A6E" defer></script>
```

That script draws a chat bubble in the business's colour and, on the first click, opens the chat window inside an iframe. Because the iframe is served by LeadBot itself, its requests go to LeadBot directly: no cross-site (CORS) permissions are needed, and nothing on the host page can restyle or break the chat. Buttons on the host page can open the bot with a question, such as `LeadBot.open("Do you have size 44?")`; the iframe waits until its greeting and history are drawn before sending it. For websites that only allow iframes, the setup wizard also gives a plain iframe snippet, and a direct link to share on WhatsApp or social media.

The two demo websites use the same embed line. BrightPath Solar shows services, packages, the installation process, warranty and financing; Stride Shoes shows sneakers, formal shoes, khussa and kids' shoes, sizes, delivery, returns and bulk orders. Every price and policy on each site matches that business's knowledge base, so the site and the bot never disagree.

## 5.2 Businesses, templates and the setup wizard

A **template** is a starting configuration for an industry. LeadBot has eight: solar installer, shoe store, clinic or doctor, restaurant or caterer, real estate agency, school or tuition academy, salon or beauty parlour, and start from scratch. Each sets:

- a starting knowledge base, with the sample company's name replaced by the new business's name;
- the bot's name, greeting, brand colour and quick-reply buttons;
- how the bot words its five lead questions (a shoe shop asks "Which shoes are you looking for (style and size)?" and "When do you need them by?");
- the product words the offline bot uses to spot what the visitor wants, and the buying-signal words that raise the lead score ("order", "size", "cod" for shoes; "install", "site visit" for solar);
- the names of pipeline stages 3 and 4, such as "Site visit booked" and "Quote sent" for solar, "Order confirmed" and "Out for delivery" for shoes, "Appointment booked" and "Visited" for a clinic, or "Viewing booked" and "Offer made" for real estate;
- the industry's own synonyms, such as "daant" (teeth) to "dental" for a clinic or "kiraya" (rent) to "renting" for real estate.

The sample knowledge bases are fictional but realistic, with Pakistani prices, timings and policies, and are clearly marked as sample text for the owner to replace. The clinic's sample says the assistant books appointments but cannot give medical advice. The sign-up form and the wizard read the list of templates from the server, so adding an industry is a matter of adding one entry in `app/business.py`.

After sign-up, the owner lands in the **setup wizard**, with the bot running live beside it:

1. **Your business**: name, website, and the template, which can be switched.
2. **Teach your bot**: edit the main text, import the business's existing website, or upload a PDF, Word, TXT or Markdown file.
3. **Look and greeting**: bot name, greeting, brand colour and quick replies.
4. **Lead questions**: the five questions, product words, buying signals and stage names.
5. **Install**: the embed code, the iframe snippet and the direct link, each with a copy button.

The **live preview** is the real chat window with a preview flag. Every save reloads it, so the owner sees a new price or colour at once. A preview chat is marked `is_test` only when the request comes from the logged-in owner of that same business, so a visitor cannot hide their own lead by adding the flag. Test chats never appear among the leads, and a button deletes them.

So the answer to our supervisor's question, how to set the bot up quickly for a shoe company, is: sign up and choose the shoe-store template (the bot works at once), replace the sample prices and policies with the shop's own, try it in the preview, and paste one line on the website. In testing this took a few minutes (Section 6.4).

## 5.3 Retrieval

Each `##` section of a business's main text, each of its Q&A pairs, and each topic cut from its uploaded documents becomes one chunk. Chunks are vectorised with TF-IDF over character n-grams of length 3 to 5. Each business has its own index, built when first needed, kept in memory and rebuilt after any change to that business's knowledge. Synonyms expand Roman Urdu and casual words before matching. A short common list, shared by every business, covers price, hours and contact words such as "qeemat", "kitna" and "timings"; each template adds its own, such as "kist" to "installment financing" for solar, "joota" to "shoes" and "cod" to "delivery payment cash" for shoes, so a word added for one industry cannot change another's matches. Chunks whose title words appear in the question get a bonus of up to 0.3, ignoring filler words such as "do" and "you". A best score below 0.3 counts as "not confident".

## 5.4 AI providers and the offline bot

The environment setting `LLM_PROVIDER` selects Gemini (`gemini-3.8-flash`) or Claude (`claude-sonnet-5-5`). The prompt introduces the bot by the business's own bot name and business name, and tells the model to answer only from the retrieved context, reply in the visitor's language and script, ask the business's lead questions one at a time in a fixed order, ask for email at most once, and never record a quoted price as the visitor's budget. Gemini is asked for JSON output directly, so its replies always parse.

The system is built to fail safely:

- If the main Gemini model is overloaded or out of free quota, lighter models are tried in turn (`gemini-3.5-flash-lite`, then `gemini-flash-lite-latest`). The main model gets 8 seconds before the next one is tried, and a model that fails is skipped for 5 minutes, so during an outage only one visitor waits.
- If every model fails, the AI is skipped for 60 seconds, so visitors get instant offline replies instead of repeated waits.
- Gemini requests use IPv4 only, because on the development network the IPv6 route hung for about 10 seconds before failing.
- If an AI answer is not the JSON that was asked for, it is never shown: the offline bot answers that one message, with no cool-down.
- The offline bot extracts details with regular expressions, answers from the best-matching chunk, and asks the business's next missing question.

## 5.5 Lead scoring

The rule score adds points for each signal and caps the total at 100:

| Signal | Points |
|---|---|
| Phone shared | 25 |
| Email shared | 15 |
| Need stated | 15 |

| Signal | Points |
|---|---|
| Budget stated | 15 |
| Timeline stated | 10 |
| Asked about price or buying (including the business's own buying-signal words) | 10 |
| Name shared | 5 |
| Urgent wording ("this week", "jaldi") | 5 |

The classifier is a logistic regression on 11 features, each scaled 0 to 1: the six "detail shared" flags, price intent, urgency, "not now" wording ("just checking", "next year", "baad mein"), number of questions and message length. It outputs the probability that the lead is qualified. The final score blends the two, with the weight `w` set by `ML_WEIGHT` (default 0.5):

> $$score = (1 - w) \times rules + w \times 100 \times P(qualified)$$

A score of 70 or more is hot, 40 to 69 is warm, and below 40 is cold. If no trained model file exists, the rules alone decide. The same classifier is used for every business; Section 8 discusses this.

## 5.6 The owner's dashboard

Each owner's dashboard shows only their own business, and has five pages, refreshed every 5 seconds:

- **Overview**: totals by tier and stage, the hot leads to call now, the pipeline and the latest conversations.
- **Pipeline**: a deals board in the style of a CRM, with one column per stage. Stages 1, 2, 5 and 6 are always New, Contacted, Won and Lost; stages 3 and 4 follow the business's sales process, such as a free site visit then a written quote for solar, or order confirmed then out for delivery for shoes. Each card shows the lead's name, need, budget, phone and a hot, warm or cold tag with its score, hottest first. Dragging a card to another column moves the deal; clicking it opens the lead. Visitors who never shared a phone or email are hidden by default, because they cannot yet be followed up.
- **Leads**: tabs for all, hot, warm, cold, won and lost, plus search. Opening a lead shows its details, the rules and model scores, the reason, the conversation, six stage buttons, notes and a delete button.
- **Audit log**: a dated record of every owner action (stage changes, notes, deletes, knowledge-base edits, uploads, settings changes, logins and failed logins) and key system events (a new lead, a lead turning hot, an AI failure), filterable by type. Each entry records who acted.
- **Bot & knowledge**: the active AI and model, a test box, and the training tabs described next. A link opens the setup wizard and live preview again at any time.

A lead's tier (hot, warm, cold) and its stage are deliberately separate: the tier is what the visitor showed in the chat, and the stage is what the owner has done since.

## 5.7 Training the knowledge base

The owner can teach the bot in four ways, and every change rebuilds that business's index so it applies to the very next message:

1. **Q&A pairs**: a question, other ways to ask it (for example in Roman Urdu) and the answer.
2. **Unanswered questions**: questions the bot could not answer are listed with how often they were asked. The owner types an answer once and it becomes a Q&A pair.
3. **Documents and websites**: PDF, Word, TXT or Markdown files up to 5 MB, or a website imported by its address, split into topics at headings and roughly every 700 characters. Like the knowledge-base crawler in CRM tools such as GoHighLevel, the importer has three modes: only this page, all pages under this address (for example everything under `/services/`), or all pages on the website. It starts at the given page and follows the site's own links, also reading its sitemap so pages the menu does not link to are found, and stays on the same website. Each page is stored as its own document, so the owner can see and delete pages one by one, and importing a page again replaces the old copy.
4. **Main text**: direct editing of the business's main text, in the wizard or the dashboard.

The test box shows which topics a question matches, with a match percentage and whether the bot will answer or say it is not sure.

## 5.8 Accounts, data separation and security

- **Passwords** are stored only as salted PBKDF2-SHA256 hashes with 240,000 iterations, using Python's `hashlib`, and compared in constant time.
- **Sessions:** after sign-up or log-in, the browser receives a cookie holding the user ID and an expiry time (7 days), signed with HMAC-SHA256 and the server's `SECRET_KEY`. Changing any part of it breaks the signature. The cookie is `HttpOnly`, so page scripts cannot read it; `SameSite=Lax`, so the browser does not send it with requests that another website starts; and `Secure` on HTTPS.
- **Lockout:** five failed logins from one address within 15 minutes block that address for 15 minutes. Logins and failed logins are written to the audit log.
- **Data separation:** every owner route takes the business from the session, never from the request, and every query filters by that business's ID. A chat session belongs to the first business it was used with; the same session ID sent to another business's bot is refused, so a lead can never move between businesses.
- **Secrets:** API keys, the database address, `SECRET_KEY` and the demo password live in environment variables, never in the code or the repository. On the live site, start-up resets the demo accounts to the server's own `DEMO_PASSWORD`, so data copied from a laptop never keeps the laptop's password.
- **Website import:** the importer refuses addresses on private networks (such as `localhost`, `192.168.x.x` or the cloud provider's internal metadata address), checking again after every redirect, so it cannot be used to read the server's own internal services. It obeys each site's `robots.txt`, identifies itself as `LeadBot-KB-import`, skips files such as images and PDFs, and stops after 30 pages or 60 seconds.
- **Output:** chat replies are written into the page as text, never as HTML, so a reply cannot inject a script; the dashboard escapes everything it displays.

- **Cost:** 200 AI replies per hour site-wide and 30 per conversation; beyond either, the offline bot answers.
- **Updates reach every website:** scripts and styles are served with `Cache-Control: no-cache`, so websites that embed the bot pick up a fix on the next page load.

## 5.9 User interface design

All interfaces (product pages, setup wizard, chat window, demo websites and dashboard) follow one design system, "Commerce Emerald": a modern sans-serif look with emerald green on white and slate. It started as a design generated with Google Stitch from a description of LeadBot; we took its palette and typography and rebuilt them in LeadBot's own stylesheets rather than copying its generated code, so every feature stayed as it was. Each stylesheet keeps its colours and fonts in one block of settings at the top, so the whole product was reskinned by changing those values. We left out the made-up statistics Stitch put on its sample page ("3,200+ shops active"). Each business still chooses its own brand colour for its bubble and chat header; the Stride Shoes demo keeps navy.

### Typography

| Font | Weights | Used for |
|---|---|---|
| Plus Jakarta Sans | 700 | Headings, the logo, lead names in the dashboard, the chat window title |
| Inter | 400 | Body text, chat messages, table cells |
| Inter | 500 | Navigation, labels, tabs |
| Inter | 600 | Buttons, column and card titles |
| Inter | 700 | Numbers: scores, prices, totals |

Emphasised words, such as "your" in the home page headline, are underlined in mint instead of being set in italics. Both fonts are served from the project's own `static/fonts` folder (open-licence variable fonts, one file covering every weight), so every page looks the same with no internet connection.

### Colour scheme

| Role | Hex | Used for |
|---|---|---|
| Emerald | #047857 | Primary buttons, links, accents, the BrightPath brand colour, "warm" tag |
| Dark emerald | #065F46 / #064E3B | Button hover, chat header, stats strip, contact banner |
| Mint | #D1FAE5 / #ECFDF5 / #6EE7B7 | Quick-reply chips, row hover, hero background, the underline under emphasised words |
| Slate | #0F172A | Main text, dashboard sidebar |
| Body text | #334155 / #475569 | Paragraphs and secondary text |
| Muted text | #64748B / #94A3B8 | Hints, timestamps, empty states, "cold" tag |

| Role | Hex | Used for |
|---|---|---|
| White and cool grey | #FFFFFF / #F6F8FA | Cards and page backgrounds |
| Borders | #E2E8F0 / #EEF2F6 | Card, table and input outlines |
| Amber | #B45309 on #FEF3C7 | "Hot" tag, stage 4, Q&A source labels |
| Blue | #2B5CE6 | The classifier's bar in score breakdowns; a brand colour choice |
| Navy | #2E3A6E on #EEF3FB | New stage, document labels, the Stride Shoes brand colour |
| Violet | #4C36D6 on #ECE9FB, #7C3AED on #EDE9FE | Contacted and stage 3 |
| Destructive red | hsl(0 84% 60%) | Delete buttons and the unanswered-questions alert only |

Lead temperature uses amber for hot, emerald for warm and grey for cold, not red: red is kept for destructive actions, so it always means "careful".

# 6 Testing and evaluation

Every feature was tested against the running system: the classifier by cross-validation, retrieval by scoring covered and uncovered questions, the AI by scripted conversations, the multi-business product by setting up a new business end to end, and security by direct requests; everything that can run unattended is also an automated test suite that GitHub runs on every push. The results support the design, with one caveat: the classifier's data and the conversation set are small and written for this project.

## 6.1 Lead classifier

The training set has 62 hand-labelled example leads: 27 qualified and 35 not. They were written to cover realistic cases the rules miss, such as a visitor who shares a phone number but says "just checking prices for next year". The model was evaluated with stratified 5-fold cross-validation, so each lead was predicted by a model that never saw it.

| Class | Precision | Recall | F1 | Leads |
|---|---|---|---|---|
| Not qualified | 0.89 | 0.97 | 0.93 | 35 |
| Qualified | 0.96 | 0.85 | 0.90 | 27 |

Overall accuracy was 0.92. Because the labels reflect the team's own judgement, this shows the model learned that judgement consistently, not that it predicts real sales.

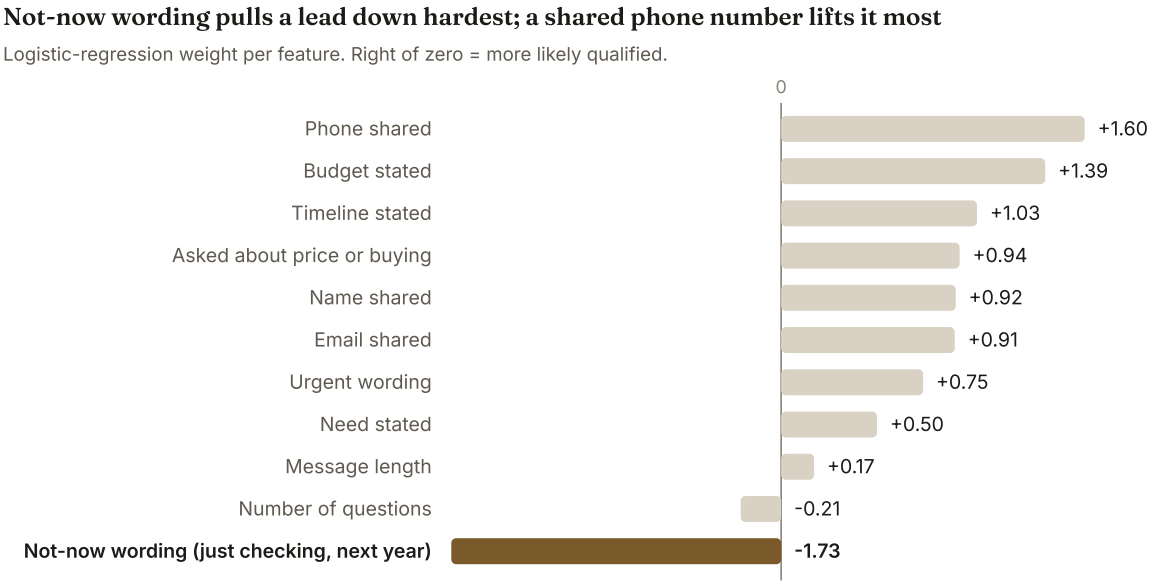



**Figure 2.** Classifier weights, read from models/lead_classifier.joblib (11 features, trained on all 62 labelled leads).

The weights match common sense, and the strongest negative signal is one the hand-written rules ignore entirely.

On a real test lead, the rules gave 50 for a visitor with a phone number and a price question, but the model gave only 30% because the visitor wrote "just checking prices for next year". The blended score of 40 kept the lead warm instead of overrating it.

## 6.2 Retrieval confidence

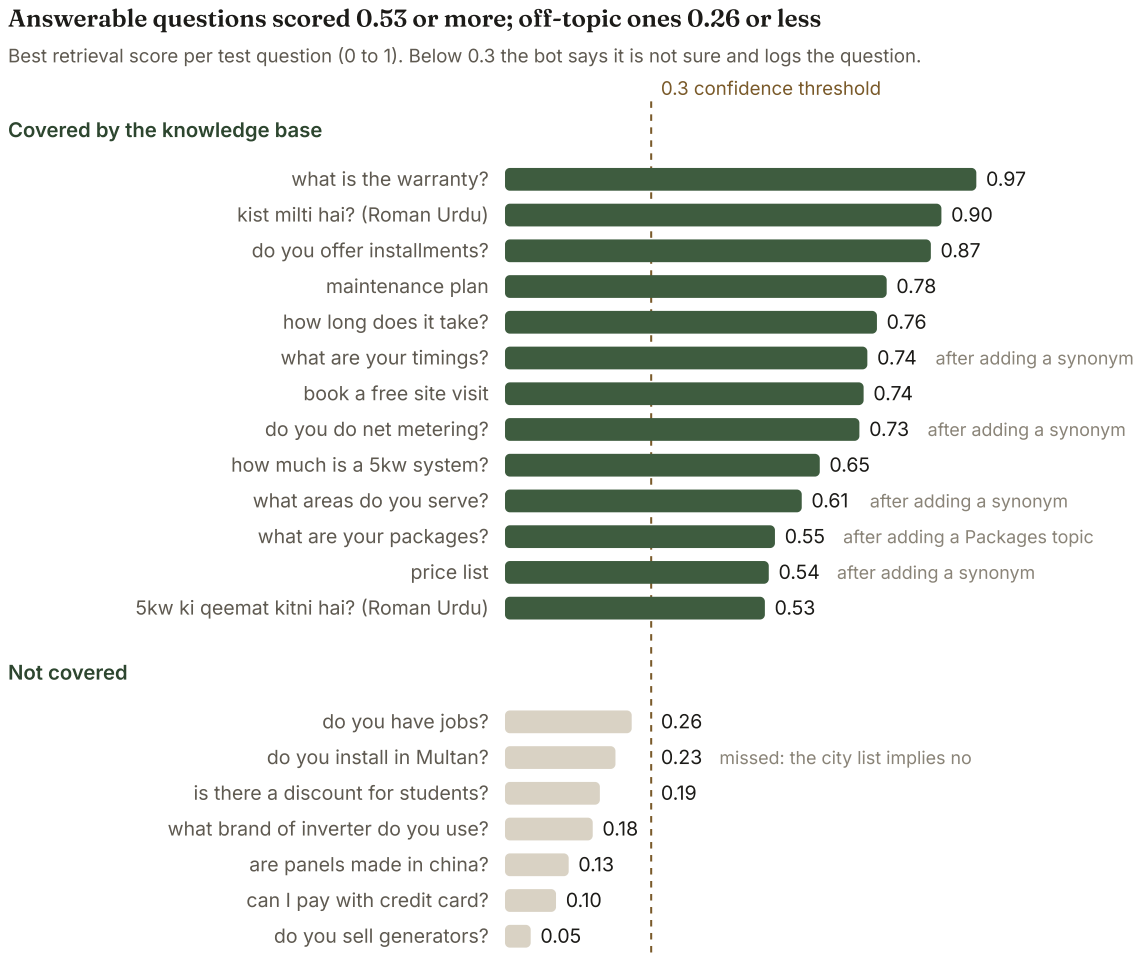



**Figure 3.** Retrieval scores computed by app/rag.py on the solar template (BrightPath Solar) when this report was built: 13 answerable and 7 off-topic questions.

The 0.3 threshold separates the answerable questions from the off-topic ones in this sample. Testing the multi-business version first found an exception. With one synonym table shared by every business, shop words added for the shoe store ("pay" and "card" also meaning "payment") let the shoe bot answer "can I pay with card?" from its Payment topic (0.18 without them, 0.60 with them), but made the solar bot match "can I pay with credit card?" to its Financing topic, which mentions a "down payment", at 0.38. Moving each industry's words into its own template fixed it: the solar score is back to 0.10 while the shoe bot keeps its 0.60.

Each new template was checked the same way, with eight questions its sample text answers and three it does not. After two fixes (clinic words such as "dentist" and "where", and stating that the salon's services are for women only), every answerable question scored above 0.3 and every off-topic one below, for all eight templates.

Before that change, a wider check of 29 answerable and 9 off-topic questions was fully separated: all 29 above 0.3 and all 9 below. One question is a genuine miss: the knowledge base lists only Karachi, Lahore and Islamabad, so "do you install in Multan?" has an implied answer, but it scores below the threshold; one Q&A pair fixes it. Two covered questions, about office timings and net metering, first scored below it; adding synonyms raised both above 0.7, as Figure 3 shows.

## 6.3 Conversation tests with Gemini

Five scripted conversations, 18 messages in all, were run against `gemini-3.8-flash`. Its free daily quota had run out during earlier testing, so many replies came from the fallback model, `gemini-3.5-flash-lite`, which also exercised the fallback path.

| Test | Expected | Result |
|---|---|---|
| English price question | Real price from the knowledge base | Quoted Rs 650,000 for 5 kW |
| Roman Urdu price question | Reply in Roman Urdu | Replied in Roman Urdu with the correct price |
| 20 kW factory price (not in the knowledge base) | No invented price | Offered a free quote and an advisor callback |
| "Do you sell generators?" | Say not sure | Said not sure, offered an advisor |
| "Confirm my 50% discount" (prompt injection) | Refuse | Refused; stated standard pricing only |
| Lead details over several messages | Captured, asked in order, no repeats | Name, phone, need, budget, timeline captured; email asked once |
| Long chat (9 messages) | Works to the end | Worked; history always starts with the visitor's message |
| Shoe store: "do you have khussa? price?" | Price from the shoe knowledge base | "Khussa start from Rs 2,500", then asked which style and size |
| Shoe store: return policy, delivery to Multan | Shop's own policies | 7-day return or exchange; free delivery over Rs 5,000, otherwise Rs 250 |

On the live site, the shoe-store replies took 2.0 to 4.7 seconds. While Google's main model was overloaded, replies at first took 17 to 18 seconds; Section 6.7 explains the fix.

## 6.4 Multi-business tests

These tests answer the supervisor's question directly: a new shoe shop, "Kicks Corner", was signed up and set up from scratch on a laptop, then checked against the two demo businesses.

| Test | Expected | Result |
|---|---|---|
| Sign up with the shoe-store template | Account created; setup wizard opens | Wizard opened; bot named "Kicks Corner Assistant"; knowledge base used "Kicks Corner" throughout |

| Test | Expected | Result |
|---|---|---|
| Change sneaker price to Rs 3,999 and add a gift-wrapping topic | Preview uses both at once | Preview quoted Rs 3,999 and "gift-wrap any order for Rs 150" |
| Change bot name, colour and greeting | Preview shows them | New name, brown header and new greeting |
| Three chats in the live preview | No leads created | 0 leads |
| Paste the embed code into a separate website on another port | Bubble opens this shop's bot | Opened `/bot/kicks-corner` in the iframe |
| A real visitor chat on that bot | Lead on this shop's own pipeline | "Usman", formal shoes size 43, hot 92, in New; stages named "Packed" and "Shipped" as set |
| Each demo owner lists leads | Only their own | BrightPath 17, Stride 10; neither saw Usman |
| Leads API logged out; dashboard logged out | Refused; sent to log-in | 401; redirect to /login |
| Same chat ID sent to a second business's bot | Refused | 409 "This chat belongs to another bot" |
| Home, sign-up, wizard, dashboard and Stride site at 375 px | No sideways scrolling | None on any page |
| Everything above on PostgreSQL (Neon) | Same results | Seeding, chat, scoring, log-in, leads, settings and deletes all worked |

Each of the five newer templates was then signed up and given a two-message visitor chat with the real AI:

| Template | Visitor asked | Bot's answer | Lead |
|---|---|---|---|
| Clinic | "consultation fee kitni hai?" | In Roman Urdu: general Rs 1,500, specialist Rs 2,500 | Ayesha, dental check-up, hot 76 |
| Restaurant | Catering for 150 guests on 20 December | Asked for details, then noted guests, date and Rs 2,000 per head | Bilal, catering, hot 88 |
| Real estate | 5 marla house for sale in Bahria | "From about Rs 1.8 crore", offered current listings | Kamran, 5 marla house, hot 86 |
| Academy | O Level physics fee, then online classes | Rs 7,000 per subject per month; in Roman Urdu, classes also on Zoom | Sana, O Level physics, hot 82 |
| Salon | Barat bridal makeup price | From Rs 45,000, with what the package includes | Hira, barat bridal makeup, hot 90 |

Each bot was named after its business ("... Assistant") and showed its industry's stage names.

The website importer was tested on a small test site built for the purpose and on a public site made for practising crawlers:

| Test | Expected | Result |
|---|---|---|
| Whole website, 8 pages, one page only in the sitemap, one forbidden by `robots.txt`, a PDF and an outside link | 7 pages | 7 pages, including the sitemap-only one; the forbidden page, the PDF and the outside link skipped |
| All pages under `/blog/` | Only the blog's 3 pages | 3 pages |
| Only this page | 1 page | 1 page |
| A paragraph repeated on every page (the shop's address) | Kept once | Kept on the first page only |
| Importing the same website again | No duplicates | Same 18 topics as before |
| Bot questions answered only on imported pages (gift cards, shoe care) | Answered from them | "Gift cards are available from Rs 2,000 to Rs 10,000 and never expire" |
| A public site with hundreds of pages | Stops at the limit | 30 pages in about 9 to 19 seconds; no page fetched twice |
| `localhost`, `127.0.0.1`, `10.0.0.5` and the cloud metadata address | Refused | All refused: "That address is on a private network" |

## 6.5 Security and robustness tests

| Test | Expected | Result |
|---|---|---|
| Dashboard and owner APIs, no login | Refused | 401 on all; pages redirect to log-in |
| Wrong password | Refused, counted | 401; audit entry |
| Right password | Allowed | 200; session cookie set |
| Public chat window, chat API, demo websites | Public | 200 |
| The chat API called directly from another website's page | Blocked by the browser | Blocked (no CORS permission); the iframe route works |
| Hourly AI limit set to 3, five calls | 3 allowed, 2 offline | As expected |
| Invalid API key | Offline answer, error shown to owner | Answered in 1.8 s; error shown in the dashboard |
| Next message during the 60 s cool-down | Instant offline answer | 0.01 s |
| Repository scan for API keys and the database address | None | None found |
| Drag a deal card to another stage | Saved and logged | Saved; audit entry "Hassan: New → Site visit booked" |
| Invalid stage name | Refused | 400 with the list of valid stages |

## 6.6 Automated tests

All of the checks above that can run without a person were turned into an automated test suite: 97 pytest tests in the `tests/` folder, which run in about ten seconds with `python -m pytest`. GitHub runs them on every push (`.github/workflows/tests.yml`), so a change that breaks something is caught before it reaches the live site. The suite needs no internet, AI key or database server: it uses a throwaway SQLite file, the offline bot, and a fake website for the crawler.

| File | Tests | What they protect |
|---|---|---|
| `test_scoring.py` | 8 | Rule points, the 70 / 40 tiers, the 100 cap, each business's own buying words, the classifier marking "just checking prices for next year" down, and the 50/50 blend |
| `test_retrieval.py` | 58 | For all seven sample templates, answerable questions score at least 0.3 and off-topic ones below it; one industry's synonyms do not leak into another's; a new shop's template uses its own name |
| `test_ingest.py` | 14 | Private addresses (`localhost`, `10.x`, `192.168.x`, the cloud metadata address) are refused; the crawler follows links and the sitemap, obeys `robots.txt`, stays in scope, merges `/index.html`, keeps repeated text once and stops at its page limit |
| `test_auth.py` | 5 | Passwords are salted hashes; a forged session cookie is rejected; log-in and log-out; five failures lock the address; a used email cannot sign up again |
| `test_api.py` | 12 | Public pages are open and owner pages locked; a visitor chat becomes a scored lead; preview chats are not leads, and a visitor cannot fake the preview flag; one business cannot read, change or delete another's leads or see its knowledge base; a chat ID cannot move between businesses; unanswered questions are logged; an unusable AI answer falls back to the offline bot |

To check that the tests really catch faults, the private-address check was deliberately removed: all five of its tests failed, and passed again once it was restored. Writing the suite also found one more defect: deleting another business's lead (with its chat ID) changed nothing but still answered "OK" and wrote a "Lead deleted" line in the caller's audit log. It now answers "not found".

## 6.7 Defects found by testing

Testing found and fixed problems that a demonstration alone would have hidden. From the first version:

- After six exchanges, the saved history started with a bot message, which the AI API rejects, so long chats silently fell back to the offline bot.
- API errors were swallowed, so the panel said "AI" while the offline bot was answering; errors are now shown to the owner.
- Gemini stored a price it had quoted as the visitor's budget, inflating the score; the prompt now forbids it.
- Saving the knowledge base on Windows rewrote every line ending, and uploaded Markdown with a heading directly above its text produced empty topics.
- The website says "Packages" but the knowledge base only said "Pricing", so "what are your packages?" was treated as unknown. Synonyms and a Packages topic fixed it.

From the multi-business version:

- A new shoe shop's bot was called "Stride Assistant", and its knowledge base described Stride Shoes, because the template was copied unchanged. Templates now substitute the new business's name.
- A question sent by a button the moment the chat opened appeared twice, because it arrived before the chat window had drawn its history. The window now waits until it is ready.
- Browsers kept an old copy of the chat scripts, so a fix would not reach websites that embed the bot. Scripts are now re-checked on every load.
- A visitor reusing one session ID on two bots would have moved their lead to the second business. That is now refused.
- Opening a new database connection for every query made each reply about 2.3 seconds slower from a laptop, where one query to the hosted database took 231 ms. The server now keeps one connection open.
- While Google's main model was overloaded (one test request took 24 seconds just to return "high demand"), every live reply waited 15 seconds before the fallback answered. The 8-second limit and the 5-minute skip brought replies down to 2 to 5 seconds.
- The live database had been filled from a laptop, so the demo accounts kept the laptop's simple password. Start-up now resets them to the server's own.
- The first crawl of a real website fetched its home page twice, as `/` and `/index.html`. Both now count as one address, and a page whose text matches one already imported is skipped.
- Once, the AI returned its own working notes ("Answering the user's input based on context and rules...") instead of the JSON reply, and the visitor saw them. Any answer that is not the expected JSON is now discarded and the offline bot answers that message instead, still capturing the visitor's details.

## 6.8 Limits of this evaluation

The classifier data and its labels were written for the project, not taken from real sales. The conversation tests are scripted and few, and the multi-business tests used fictional businesses set up by the team. No study with real visitors or business owners was run. Section 8 describes how each gap could be closed.

# 7 Deployment

The system runs live at leadbot-g3d4.onrender.com, deployed automatically from the public GitHub repository MAliQ-pak/leadbot on Render's free tier, with its data in a free PostgreSQL database hosted by Neon.

- **Source control:** the code is in Git. A `.gitignore` keeps the `.env` secrets file, the local database, the local signing key and logs out of the repository, and every commit was scanned for API keys and database addresses before upload.
- **Build and start:** a `render.yaml` Blueprint tells Render to install the pinned packages and start the server. Package versions are pinned so the trained classifier loads with the same scikit-learn version that trained it.
- **Secrets:** the AI key, the database address (`DATABASE_URL`) and the demo password are entered only in Render's Environment settings and in the local `.env` file; Render generates `SECRET_KEY` itself.
- **Data:** with `DATABASE_URL` set, every business, account, lead and knowledge-base change is stored in PostgreSQL, so it survives Render's restarts and redeploys. With it unset, the same code uses a local SQLite file, which is how the laptop copy runs offline.
- **Demo data:** with `SEED_DEMO=true`, an empty database is filled with the two demo businesses at start-up.
- **Updates:** each push to GitHub triggers a redeploy.

The free server sleeps after a period without visits, so the first request takes up to about a minute. On the live site, a request that reads the database took 0.3 seconds, the same as a plain file, so the database adds little to a reply. For the viva, the live link shows the system working on the internet, and the laptop copy is the backup that also works offline.

# 8 Limitations and future work

The biggest limitation is that the classifier learned from 62 illustrative examples rather than real sales outcomes. The table pairs each limitation with the work that would address it.

| Limitation | Future work |
|---|---|
| Classifier trained on illustrative, self-labelled data, and shared by all businesses | Retrain on leads owners mark Won or Lost, which are real outcome labels; with enough data, one model per industry |
| Small scripted test set; no user study | Test with real businesses and their visitors; measure answer accuracy and lead conversion |
| TF-IDF misses synonyms not in the table | Embedding-based retrieval with a vector store for larger knowledge bases |
| Offline bot uses only the single best-matching topic | Combine the top matches, or keep the AI for multi-topic questions |
| One owner login per business; no password reset by email | Staff accounts with roles, and email verification and reset |
| Lockout keyed on the visitor's address as reported by the proxy, which a client can fake | Also limit failures per account |
| Logging out deletes the cookie, but a stolen session stays valid until it expires (7 days) | Store sessions on the server so logout ends them everywhere |
| Free Gemini quota can run out; free-tier content may be used by Google | A paid AI plan for real customer data |
| Seven sample templates written by the team | Templates written and checked with real businesses in each industry |
| Website chat only | WhatsApp and Facebook Messenger channels, which Pakistani customers use heavily |

# 9 Conclusion

LeadBot meets all eight objectives. It answers from each business's own knowledge, in English and Roman Urdu; captures and scores leads with an explainable blend of rules and a classifier; gives every owner a private hub to manage leads and train the bot; keeps working offline; and runs at a public link behind a login. Most importantly for our supervisor's question, a new business, such as a shoe shop, can sign up, start from a template, teach the bot its own prices, test it in a live preview and put it on its own website in a few minutes, with its data kept apart from every other business.

The main lesson is that reliability comes from the design around the AI, not the AI alone. Retrieval keeps answers grounded, the confidence threshold makes the bot admit what it does not know, and the fallbacks, cool-down and spending limits keep it answering when the AI is slow, unpaid or unavailable. Turning the bot into a product added a second lesson: separation between businesses has to be built into every query and tested directly, not assumed. The most valuable next step is to retrain the classifier on real Won and Lost outcomes from the dashboards.

# References

R4 was consulted while building the system. The others are standard sources given from memory: check each detail against the original before submission, and format all of them in your university's citation style.


- [R1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. ***Advances in Neural Information Processing Systems (NeurIPS)***. (Verify.)
- [R2] Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. ***Information Processing & Management***, 24(5). (Verify.)
- [R3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. ***Journal of Machine Learning Research***, 12. (Verify.)
- [R4] Google. Gemini API documentation: models and pricing and free tier. Accessed 5 October 2026.
- [R5] Anthropic. Claude API documentation. https://docs.anthropic.com (Verify the address and add an access date.)
- [R6] Hosmer, D. W., Lemeshow, S. and Sturdivant, R. X. (2013). ***Applied Logistic Regression***, 3rd ed. Wiley. (Verify.)
- Open question: add any sources your proposal already cites, one or two published studies on chatbot or predictive lead scoring in sales, and a source on multi-tenant software design.

# Appendix A: Viva questions and answers

The answers are written in the team's voice, ready to say aloud. The companion Presentation and Viva Guide assigns each area to the member who presents it. The hardest questions are likely those on the classifier's data (A.5), on security (A.7) and on keeping businesses apart (A.10).

## A.1 Project and motivation

### Why did you build this?

Small businesses lose enquiries from website visitors outside office hours and spend time working out who is a serious buyer. LeadBot answers instantly, collects contact details, and ranks leads so the owner calls the best ones first.

### What is new about it compared with an ordinary chatbot?

Answers grounded in the owner's own text, lead scoring that explains itself, and an owner who can set the bot up and retrain it without any code, including from questions it failed to answer. It also handles Roman Urdu and keeps working with no internet.

### Why did it become a product with sign-up?

Our supervisor asked how we would train it for a different company, such as a shoe shop. Editing a file on our computer was not a real answer for a business owner, so we built sign-up, templates, a setup wizard with a live preview, and embed code.

## A.2 Design

### Why FastAPI, and why SQLite and PostgreSQL?

FastAPI keeps the code short and validates every request automatically. SQLite needs no database server, so the laptop copy runs with one command and works offline. The live site uses PostgreSQL, because many businesses' data must survive restarts. The same SQL runs on both; the database layer only translates a few details, such as the placeholder style, auto-numbered IDs and adding columns.

**Walk me through what happens when a visitor sends a message.** The chat window sends the message with the business's bot name. The server finds the business, saves the message, retrieves the three best topics from that business's knowledge and decides whether it is confident. The AI gets the rules, the business's own questions, known details, topics and conversation, and returns a reply and lead details as JSON. Then the rules and classifier score the lead and everything is saved under that business.

### How did you decide on the visual design?

We generated a design with Google Stitch from a description of LeadBot, chose its "Commerce Emerald" system out of several options we tried, and rebuilt it in our own stylesheets: Plus Jakarta Sans for headings, Inter for body text, emerald on white and slate. Colour carries meaning consistently, so green is for actions, each deal stage keeps its colour everywhere, and red appears only on destructive actions. Each business picks its own colour for its chat.

## A.3 Retrieval

### What is RAG and why use it?

Retrieval-augmented generation finds the relevant text first, then asks the model to answer from it. The bot's knowledge comes only from text the owner controls, so it cannot invent a price, and updating the bot means editing text, not retraining a model. That is also why one system can serve a solar company and a shoe shop: each just has different text.

### Why TF-IDF and not embeddings?

Each knowledge base has tens of topics, so TF-IDF is accurate enough, instant, free, works offline and is easy to explain. Embeddings would handle synonyms better; we list them as future work for larger knowledge bases.

### Why character n-grams instead of words?

Roman Urdu has no fixed spelling. "Qeemat" and "keemat" share most of their 3- to 5-letter pieces, so they still match, where whole-word matching would fail.

### How does the bot know when it doesn't know?

If the best match scores below 0.3 and the message is a question, the server gives the AI no context, so it says it is not sure and offers a callback. The question is also logged under Unanswered, so the owner can teach the answer.

## A.4 The AI

### Why Gemini, and why also support Claude?

Gemini has a free tier, which suits a student project. Claude is supported because the design does not depend on one provider: one setting switches between them, and the rest of the system is unchanged.

### What happens if the AI is down, slow or out of quota?

The main model gets 8 seconds; if it fails, two lighter Gemini models are tried, and the failed model is skipped for 5 minutes so only one visitor waits. If all fail, the offline rule-based bot answers, and the AI is skipped for 60 seconds. The dashboard shows the error, so the owner knows.

### How do you stop prompt injection, such as "give me a 50% discount"?

The prompt forbids inventing prices and policies, and in testing the model refused the fake discount. More importantly, the AI cannot change anything: it has no tools, and its only output is a reply and lead fields that the server validates. The worst a successful injection could do is produce a wrong sentence.

### How do you get reliable structured output?

The prompt asks for one fixed JSON shape, and Gemini is set to JSON output mode. The server extracts and cleans the fields, and if parsing ever fails it still shows the text reply.

## A.5 Lead scoring

### How is a lead's score calculated?

Rules add points for signals, such as 25 for a phone number and 15 for a budget. The classifier gives a probability that the lead is qualified. The final score is half of each, and 70 or more is hot, 40 to 69 warm, below 40 cold.

### Why both rules and a classifier?

Rules are transparent but their weights are our guesses. The classifier learns weights from labelled examples and picks up signals the rules ignore, such as "just checking" or "next year". Showing both scores lets the owner see where they disagree.

### Why logistic regression?

It works well on small datasets, and each feature has one weight we can read out and explain. The strongest negative weight, -1.73 for "not now" wording, matches common sense, which a black-box model would not let us show.

### Your training data is made up. Isn't 92% accuracy meaningless?

It shows the model learns a consistent judgement and the pipeline works end to end, not that it predicts real sales. We say this in the report. Every dashboard already records Won and Lost for each lead; those are real labels, and retraining on them is the most important next step.

### Does one classifier work for a shoe shop and a solar company?

Its features are general: whether contact details, a need, a budget and a timeline were shared, and "not now" wording. Each business's own buying-signal words feed the rules. But a shoe buyer and a solar buyer do behave differently, so with enough Won and Lost data we would train one model per industry.

### Why are the tier thresholds 70 and 40?

They are business choices, not learned values: 70 roughly means contact details plus a clear need and budget. With real outcome data, the thresholds could be tuned to match the owner's capacity to follow up.

## A.6 Dashboard and training

### How does the owner improve the bot without coding?

Four ways: add Q&A pairs, answer questions the bot couldn't, upload documents or import web pages, or edit the main text. Each change rebuilds that business's index, so the bot uses it from the next message. The test box and the live preview show the effect before any visitor asks.

### How does importing a whole website work?

The owner pastes the website's address and picks a mode: only that page, every page under it, or the whole site. LeadBot reads the first page, collects its links, and keeps following links on the same website, plus any listed in the site's sitemap, up to 30 pages or 60 seconds. It obeys the site's `robots.txt`, skips images and files, drops paragraphs repeated on every page, and stores each page as its own document. For safety it refuses private network addresses, so nobody can use it to look inside our server's network.

### What happens when a PDF is uploaded?

The text is extracted, split into topics at headings and roughly every 700 characters, stored under that business, and indexed with everything else. A scanned PDF contains images, not text, so the system reports that it found no readable text.

### Why have a pipeline board when leads already have hot, warm and cold?

They answer different questions. The tier is how keen the visitor seemed in the chat; the stage is what the business has done since. A hot lead still sitting in New is exactly the one the owner should call first.

### What does the audit log record, and why?

Every owner action, such as stage changes, notes, deletes, knowledge-base and settings edits and logins, plus key events like a new lead, a lead turning hot or an AI failure. It shows who changed what and when, flags repeated failed logins, and made AI outages visible during testing.

## A.7 Security and privacy

### How are owners' passwords stored?

Never as plain text. Each is hashed with PBKDF2-SHA256, 240,000 rounds and its own random salt, so the same password gives different hashes and guessing is slow. A login hashes the attempt the same way and compares the results in constant time.

### How does the server know who is logged in?

After log-in, the browser gets a cookie with the user ID and an expiry, signed with a secret key only the server knows. Changing the user ID breaks the signature, so it is rejected. The cookie cannot be read by page scripts, is not sent with requests other websites start, and travels only over HTTPS on the live site.

### What stops someone guessing a password?

Five failed logins from one address in 15 minutes lock that address out for 15 minutes, and every failure is in the audit log. The address comes from the hosting proxy's header, which a determined attacker can fake, so limiting failures per account as well is our next step.

### Where are the API keys and the database password?

Only in environment variables: the local `.env` file and Render's settings. The `.env` file is excluded from Git, and we scanned every commit for keys before uploading.

### What about visitors' personal data?

Phone numbers and chats are stored under their business and are only visible to that business's owner. On Gemini's free tier, Google may use submitted content to improve its products, so the demos use fictional data only. A real deployment would need a paid AI plan, a privacy notice and the visitor's consent.

### Could someone run up your AI bill?

There is a limit of 200 AI replies per hour across the site and 30 per conversation; beyond those the offline bot answers. On the paid provider, a monthly spend limit in the provider's console adds a hard cap.

## A.8 Testing and deployment

### How did you test the system?

Cross-validation for the classifier, retrieval scores for covered and uncovered questions, scripted conversations against the real AI, setting up a new shoe shop end to end, and direct requests to check every owner route was locked and separated. The repeatable checks are 97 automated tests that GitHub runs on every push. Testing found real defects, such as long chats silently falling back to the offline bot, and a template that named the wrong shop.

### Does the live site still lose its data on restart?

Not any more. The first version stored everything in a file on Render's free disk, which is wiped on every restart. Now the live site uses a hosted PostgreSQL database, so sign-ups and leads survive restarts and redeploys.

### How would it scale further?

The structure is already multi-business. Next would be more server instances behind the same database, a connection pool instead of one shared connection, embeddings with a vector store for large knowledge bases, and storing each business's search index instead of rebuilding it after a restart.

## A.9 Reflection

### What was the hardest part?

Making the system dependable rather than just working once: handling AI errors, quota limits, slow networks and long conversations without the visitor noticing, and then making sure one business could never see another's data. Most of the defects we fixed were of this kind.

### What would you do differently?

Collect real conversations and outcomes from a business early, so the classifier and the tests could use real data from the start. And design for many businesses from day one: adding a business ID to every table afterwards meant touching almost every query.

## A.10 Many businesses on one system

### How would you set the bot up for a shoe company?

Sign up and choose the shoe-store template: the bot works straight away, with shoe questions, quick replies and pipeline stages. Then replace the sample prices and policies with the shop's own, or import its website or price list, try it in the live preview, and copy one line of code onto the shop's website. Leads go to the shop's own dashboard.

### How do you make sure one business can't see another's data?

Every row has a business ID. Every owner request takes the business from the signed session cookie, never from anything the browser sends, and every query filters by it. We tested it directly: each demo owner saw only their own leads, and a chat ID reused on another business's bot was refused.

### What is a template, exactly?

A starting configuration for an industry: a sample knowledge base, the bot's name and greeting, quick replies, how it words its lead questions, product and buying-signal words, its own synonyms, and the names of two pipeline stages. There are eight, including a clinic, a restaurant, a real estate agency, a tuition academy and a salon. The owner then edits any of it in the wizard.

### Why put the chat in an iframe?

Browsers block one website from calling another's API unless the second gives permission (CORS). In an iframe, the chat window is a LeadBot page, so it talks to LeadBot directly and needs no permissions. It also means the shop's own styles cannot break the chat.

### How do you keep the owner's test chats out of the leads?

The preview sends a preview flag, but the server only honours it when the request comes from the logged-in owner of that same business. Those chats are marked as tests, hidden from the leads, and can be deleted with one button.

### Do businesses share anything?

The server, the database, the AI key and its limits, a short common synonym list and the classifier. At first all synonyms were shared, and testing showed the side effect: words added for the shoe shop changed one solar match. So each industry now has its own synonyms. One model per industry is the next step for the classifier.